

BITÁCORA DE PROMPTS

Proyecto: FreelanceFlow

Aplicación Proyecto SI P1 - Caso 4

Sistemas de Información - Ingeniería Comercial

Universidad Católica del Norte - Sede Coquimbo

Profesores: Boris Bugueño - Alejandro Paolini

Junio 2026

1. Introducción

Este documento registra la estrategia de Prompt Engineering utilizada durante el desarrollo del proyecto FreelanceFlow, una aplicación web de gestión y control financiero para freelancers. Se documentan los prompts más relevantes utilizados en las herramientas de IA: Google AI Studio, Claude (Anthropic), Stitch y Gemini, evidenciando el proceso iterativo de desarrollo asistido por inteligencia artificial.

2. Estrategia de Prompt Engineering

La estrategia utilizada se basó en los siguientes principios:

- 1. Prompts descriptivos y específicos: cada instrucción incluía contexto del proyecto, tecnología utilizada y resultado esperado.
- 2. Iteración progresiva: cuando el resultado no era el esperado, se refinaba el prompt con más detalles o correcciones específicas.
- 3. División por fases: se separaron los prompts por etapa del proyecto (diseño, desarrollo, despliegue).

3. Fase 1: Descubrimiento Notebook LM

3.1 Prompt con Gemini para Notebook LM

Primera interacción con la IA, para identificar si los resultados son mas completos que solo haciendo "Deep Resources" en Notebook LM.

Herramienta	Detalle
Gemini	Prompt: <i>[necesito que me ayudes a crear un prompt para poder empezar con la investigación en notebook LM, porque necesito crear una aplicación que resuelva la problemática de la imagen que es el caso que le toco a mi grupo y en el pdf están "el desarrollo del trabajo completo", pero primero necesito que me ayudes con la búsqueda de información para poder crear el reporte]</i>

3.2 Prompt para Notebook LM

El resultado que me genero el Gemini se aplica a notebook

Herramienta	Detalle
Notebook LM	Prompt: <i>[Actúa como un Consultor Tecnológico Senior y Analista de Sistemas. Utilizando la función Deep Research y basándote exclusivamente en los documentos cargados (el caso "Freelance en Apuros: La vida desordenada de Lucía" y la guía de "Desarrollo de Soluciones Tecnológicas Asistidas por IA"), redacta de forma completa, formal y exhaustiva el INFORME EJECUTIVO: DOCUMENTO DE ESPECIFICACIÓN DEL SISTEMA para la Fase 1 del proyecto.</i> <i>El documento debe estar redactado en español profesional, listo para ser descargado y entregado, estructurado estrictamente bajo los siguientes capítulos y secciones:</i> --- # INFORME EJECUTIVO: ESPECIFICACIÓN DEL SISTEMA ## PROYECTO: DEL CAOS MANUAL A LA SOLUCIÓN DIGITAL (CASO: FREELANCE EN APUROS) ### 1. INTRODUCCIÓN Y CONTEXTO DEL NEGOCIO - 1.1. <i>Resumen del Caso: Describe la situación actual de Lucía como diseñadora gráfica independiente, su volumen de clientes (fijos y esporádicos) y sus modalidades de cobro (por proyecto, por hora y por suscripción).</i> - 1.2. <i>Misión del Proyecto: Define el objetivo principal del grupo (centralizar clientes, proyectos, horas trabajadas y pagos mediante un dashboard financiero y una aplicación móvil con cronómetro).</i> ### 2. DIAGNÓSTICO DE LA SITUACIÓN ACTUAL Y DOLORES DEL CLIENTE (PAIN POINTS) - 2.1. <i>Análisis del Caos Manual: Detalla las falencias y riesgos de utilizar herramientas aisladas (Word para facturas, notas del celular para horas, y un Excel desactualizado hace dos meses).</i> - 2.2. <i>Impacto Financiero y Operativo: Analiza las consecuencias críticas detectadas, específicamente la incapacidad de demostrar legal o financieramente las deudas pendientes de clientes desde enero.</i> ### 3. ESPECIFICACIÓN DE REQUERIMIENTOS FUNCIONALES (RF) <i>Describe en detalle cada requerimiento indicando qué debe hacer el software para resolver el problema. Clasifícalos obligatoriamente de la siguiente manera:</i> - RF-01: <i>Módulo de Dashboard Financiero - Capacidad de calcular y mostrar visualmente cuánto se ha facturado en el mes actual y el monto exacto de las cuentas por cobrar (deudas).</i> - RF-02: <i>Módulo de Analítica de Clientes - Capacidad de procesar y cruzar datos para identificar qué cliente genera el mayor ingreso financiero y cuál consume la mayor cantidad de horas de trabajo.</i> - RF-03: <i>Módulo de Cálculo de Tarifa Efectiva Real - Algoritmo o lógica matemática para calcular la tarifa por hora real de Lucía (Ingresos percibidos de un cliente divididos por las horas reales cronometradas).</i> - RF-04: <i>Módulo de Cronómetro (Tracker de Tiempo) - Funcionalidad en la app sencilla para iniciar (Start) y detener (Stop) un temporizador que registre el tiempo exacto trabajado, vinculándolo automáticamente a un cliente y proyecto específico.</i>

	- RF-05: Módulo de Gestión de Entidades - Capacidad del sistema para crear, leer, actualizar y eliminar (CRUD) registros de Clientes, Proyectos y Pagos. #### 4. ESPECIFICACIÓN DE REQUERIMIENTOS NO FUNCIONALES (RNF) Define las restricciones y atributos de calidad que el sistema debe cumplir obligatoriamente según el contexto de Lucía y el proyecto: - RNF-01: Usabilidad e Interfaz (UI/UX) - La interfaz debe ser intuitiva, limpia y de fácil adopción para una usuaria con hábitos desordenados de gestión. - RNF-02: Arquitectura y Despliegue Dual - El sistema debe soportar un entorno web para el dashboard financiero personal y una interfaz simplificada/móvil para el uso diario del cronómetro. - RNF-03: Seguridad y Consistencia de Datos - Mecanismos para asegurar que los registros de horas y pagos no se pierdan o alteren, garantizando la integridad de la base de datos SQL relacional. #### 5. PROPUESTA DE ARQUITECTURA TECNOLÓGICA GENERAL - 5.1. Componentes del Sistema: Explica conceptualmente cómo interactuarán las tres capas requeridas: la interfaz de usuario (basada en los diseños interactivos que se realizarán en Stitch), la lógica del servidor (Back-end) y el almacenamiento persistente (Base de Datos Relacional SQL). - 5.2. Justificación de la Solución: Explica por qué una arquitectura integrada y relacional resuelve de forma definitiva el desorden operativo de Lucía frente a su ecosistema anterior de Word y Excel. --- Por favor, expande cada punto con explicaciones detalladas, lenguaje técnico adecuado para Ingeniería Comercial y Sistemas de Información, omitiendo marcadores de posición (placeholders) o resúmenes vagos. Entrega el documento listo para su revisión y exportación.]
--	--

4. Fase 1: Prototipado con Stitch

4.1 Diseño de pantallas principales

Prompts utilizados en Google Stitch para generar el prototipo interactivo de FreelanceFlow:

Herramienta	Detalle
Google Stitch	Prompt: [Se Copio y pego el informe ejecutivo generado con notebook LM] Objetivo: Generar el prototipo visual de la pantalla de inicio de FreelanceFlow Resultado: Prototipo interactivo con pantallas de login, dashboard y gestión de proyectos
Google Stitch	Prompt: [Aplica las siguientes directrices de Idioma, Formato Financiero, Estilo Visual y UI/UX para el diseño del prototipo interactivo en Stitch. El diseño debe ser sofisticado, limpio y optimizado para una visualización de datos eficiente (Dashboard-driven), emulando un producto SaaS profesional moderno. 1. LOCALIZACIÓN E IDIOMA (Espacio de Trabajo 100% en Español y CLP): • Idioma de la Interfaz: Toda la aplicación, etiquetas, menús, botones y alertas deben estar redactados estrictamente en español. (Ej:

Cambiar "Dashboard" por "Tablero de Control" o "Panel Financiero", "Clients" por "Clientes", "Invoices" por "Facturas", etc.).

- **Formato de Moneda:** Todos los valores monetarios, ingresos, deudas y tarifas deben expresarse exclusivamente en Pesos Chilenos (CLP), utilizando el símbolo "\$" antes del número y configurando montos con separador de miles por puntos sin decimales (Ejemplo: \$1.250.000, \$85.000, \$450.000).
 - **Nomenclatura Financiera Local:** Utiliza términos del contexto chileno en el dashboard, tales como "Monto Facturado", "Cuentas por Cobrar" (para las deudas de clientes), "Tarifa Efectiva por Hora" e "Ingresos Mensuales".
2. **PALETA DE COLORES** (Enfoque "Charcoal Tech & Creative Accent"):
- **Color Primario (Fondo Principal/Estructura):** Blanco puro (#FFFFFF) y Gris ultra-claro (#F8F9FA) para las secciones y contenedores. Genera un lienzo limpio que da sensación de orden absoluto.
 - **Color Secundario (Texto y Navegación):** Gris Oscuro/Antracita Profundo (#1A1D20). Transmite una estética de desarrollo de software premium (estilo Stripe o Linear).
 - **Color de Acento (Creatividad y Acción):** Azul Eléctrico o Índigo Vibrante (#3B82F6 o #4F46E5). Se utilizará EXCLUSIVAMENTE para los elementos de acción principal: el botón de "Iniciar Cronómetro", enlaces activos de navegación y botones de envío.
 - **Color de Alerta/Estado:** Verde Esmeralda (#10B981) para pagos recibidos y facturas al día; Rojo Coral (#EF4444) para cobros pendientes o alertas críticas de clientes con deudas desde enero.
3. **TIPOGRAFÍA Y JERARQUÍA** (Estilo "Clean-Tech"):
- **Fuente:** Inter, SF Pro o Arial en variantes Regular, Medium y Bold.
 - **Jerarquía:** Títulos de secciones grandes y en negrita (#1A1D20), textos secundarios o etiquetas de datos en Gris Neutro (#6B7280). Las métricas financieras clave en CLP deben tener un peso visual prominente (Bold y tamaño destacado).
4. **ESTILO DE COMPONENTES** (UI Elements):
- **Contenedores (Cards):** Bordes sutilmente redondeados (8px a 12px) con sombras muy ligeras (Drop Shadow suave: rgba(0, 0, 0, 0.05)) para separar los bloques de información.
 - **Botón del Cronómetro (Módulo Tracker):** Debe ser un componente de alta prioridad visual en la vista móvil. Cuando esté inactivo, fondo gris claro con icono de 'Play'. Al estar activo, debe cambiar a un estado pulsante o con el color de acento de fondo e icono de 'Pausa', mostrando un contador de tiempo digital (HH:MM:SS).
 - **Tablas y Gráficos (Dashboard):** Líneas de división ultra-delgadas en gris claro (#E5E7EB). Los gráficos de barra o de dona (para ver la rentabilidad de clientes en CLP y horas) deben usar variaciones tonales de la paleta principal.
5. **FILOSOFÍA DE DISEÑO:**
- **Minimalismo Funcional:** Espacio en blanco amplio entre componentes. El caos operativo de Lucía se soluciona visualmente

]

5. Fase 2: Desarrollo con Google AI Studio

5.1 Inicialización del proyecto

Prompts utilizados en Google AI Studio para generar el código base de la aplicación:

Herramienta	Detalle
Google AI Studio	Prompt: <i>[Necesito crear una aplicación web responsive, moderna y fácil de usar orientada a la gestión financiera y de tiempo para una diseñadora freelance (Caso: "Freelance en Apuros"). El sistema debe actuar como un "Panel de Control Financiero y Tracker de Tiempo Integrado" que procese los datos del negocio y resuelva el caos operativo. La aplicación debe estructurarse en una sola página (Single Page Application) o pestañas claras y contener los siguientes componentes clave en su interfaz y lógica:</i> MÓDULO DE INGESTA DE DATOS (ÁREAS DE ENTRADA): - Un área interactiva (Drag and Drop / Arrastrar y soltar) o un formulario limpio que permita cargar o simular la entrada de datos de: Clientes, Proyectos, Pagos Recibidos (en CLP) y Registros de Horas. - Un componente visual de "Cronómetro/Temporizador" en tiempo real (Tracker con botones de Iniciar y Detener) que registre las horas trabajadas asociadas a un cliente específico. Al detenerse, debe calcular el tiempo exacto en formato HH:MM:SS y simular su almacenamiento. LOGICA DE NEGOCIO Y PROCESAMIENTO (CÁLCULOS): <i>Al presionar un botón de "Actualizar Panel" o "Analizar Datos", el sistema debe procesar la información internamente para responder obligatoriamente a tres preguntas de negocio:</i> Pregunta 1 (Balance Mensual): Calcular de forma dinámica el Monto Total Facturado en el mes actual frente al Monto Total de Cuentas por Cobrar (Deudas pendientes en CLP). Pregunta 2 (Rentabilidad de Clientes): Realizar un cruce de datos para identificar y destacar cuál es el cliente que genera más ingresos en dinero (\$ CLP) y cuál es el cliente que consume más horas de trabajo. Pregunta 3 (Tarifa Efectiva Real): Calcular la métrica de eficiencia dividiendo el total de ingresos percibidos por un cliente entre las horas reales cronometradas para ese mismo cliente. INTERFAZ DEL DASHBOARD (VISUALIZACIÓN): - El diseño debe ser ultra-limpio (estilo SaaS moderno) utilizando una paleta de colores basada en blanco, gris oscuro (#1A1D20) y un azul eléctrico (#3B82F6) para los botones de acción como el cronómetro. - Debe incluir tarjetas resumen (KPI Cards) de alto impacto visual para las métricas en Pesos Chilenos (CLP, ejemplo: \$1.500.000) con formato de puntos como separador de miles. - Debe usar componentes visuales (como barras de progreso o alertas en Verde para al día y Rojo para deudas) que dejen en evidencia inmediata qué clientes tienen pagos pendientes desde enero, permitiendo a Lucía demostrar visualmente la deuda. <i>Genera el código estructurado (puedes usar HTML, CSS con Tailwind para el diseño responsive, y JavaScript puro para la lógica de los cálculos y el cronómetro) asegurando que la interfaz sea elegante, minimalista y profesional para presentar ante un comité evaluador de sistemas.</i> Objetivo: Crear la estructura base del proyecto FreelanceFlow con React, Vite y Tailwind CSS Resultado: Proyecto inicializado con estructura de carpetas, componentes base y configuración de rutas
	Por favor, mantén intacta toda la estructura de código, las funciones lógicas de JavaScript/TypeScript, los archivos y los componentes actuales de la aplicación. NO reestructures el proyecto. Sin embargo, necesito que cambies por completo el estilo visual (las clases de Tailwind CSS) de toda la aplicación. Actualmente se ve muy al estilo de un videojuego y quiero cambiarlo radicalmente para que adopte la estética limpia, minimalista y profesional de nuestro diseño en Stitch ("FreelanceFlow"). Por favor, modifica los archivos de estilo (como index.css o las clases de diseño en App.tsx) aplicando únicamente estas reglas estéticas: PALETA DE COLORES REFINADA: Fondos principales: Cambia cualquier fondo llamativo por blanco puro (#FFFFFF) o un gris ultra-claro (#F8F9FA) para dar sensación de orden. Textos: Elimina tipografías de estilo juego. Usa fuentes limpias (como Inter o sans-serif) y cambia el color del texto principal a gris oscuro/antracita (#1A1D20) y los textos secundarios a gris neutro (#6B7280). Botones y Acentos: Quita las sombras pesadas de videojuego. Todos los botones de acción principal deben ser de color Azul Índigo Vibrante (#4F46E5). Estados de Alerta: Las deudas o saldos vencidos deben resaltar sutilmente en color Rojo Coral (#EF4444). BORDES Y SOMBRAS PROFESIONALES (Estilo SaaS): - Elimina los bordes negros gruesos (estilo 'border-2 border-black' o similares si los hay).

	• Aplica bordes redondeados suaves y modernos a las tarjetas financieras (usa la clase 'rounded-xl' o 'rounded-2xl' de Tailwind). • Reemplaza las sombras duras por sombras ultra-ligeras y profesionales (usa 'shadow-sm' o 'shadow-md' suaves). 1. ESPACIADO (UI/UX Despejada): • Dale más "aire" a los componentes aumentando los márgenes y paddings de forma sutil, eliminando la saturación visual para que el panel se vea ejecutivo, ordenado y limpio para Lucía.
	manteniendo todo lo demas constante, necesito que me hagas ciertos cambios, 1) que quites los mensajes que son como explicativos del caso porque tenemos que entregar la aplicación en blanco el docente solo tendra que subir un archivo y ver que funcione la aplicación, 2) en el ingreso de datos de manera manual cuando seleccionamos la especialidad/categoría de la empresa puedes colocar varios opciones de rubros y añadir una opción para hacer una escritura manual de ser necesario, 3) en el cobro/ pago haz que haya una opción que el monto se haga a raíz de las horas que se trabajo y el precio de tasa referencial o por las tarifas por hora y que tambien tenga la opción de colocar un monto manual, 4) agrega un filtro en el balance de deudas, que los datos se puedan ver de manera mensual, trimestral y anual 5) quiero que agregues algo en donde yo pueda descargar un excel o algun archivo compatible con computador o tablet donde pueda descargar ese archivo con la actualizacion de los datos de los clientes, proyectos y todo el diagnostico
	Intentamos colocar un inicio de sesión con correo pero se veía muy raro, así que desistí de la idea.

6. Fase 3: Resolución de errores con Claude

6.1 Error de host bloqueado en Render

Durante el despliegue en Render se presentó un error de host no permitido. Se utilizó Claude para diagnosticar y resolver el problema:

Herramienta	Detalle
Claude (Anthropic)	Prompt: <i>contexto, realice un proyecto con google AI que tuve que enlazarlo con github, y ese github lo tuve que enlazar con un render, el render me dice que está vivo (live) pero me meto al link y me sale ese error [Blocked request. This host is not allowed], como puedo arreglarlo</i> Objetivo: Resolver el error de host bloqueado en el despliegue de Render Resultado: Se identificaron dos soluciones: agregar allowedHosts en vite.config.ts y configurar build de producción con npx serve dist

6.2 Configuración de build de producción

Herramienta	Detalle
Claude (Anthropic)	Prompt: <i>en la segunda opción cuando me dices instalar sirve a que te refieres, donde se hace eso? soy ingeniera comercial y no sé nada de programación pero quiero hacer la segunda solución</i> Objetivo: Entender cómo configurar el servidor de producción sin conocimientos técnicos previos Resultado: Guía paso a paso para modificar package.json en GitHub y configurar Build Command y Start Command en Render

6.3 Mejora del inicio de sesión

Herramienta	Detalle
Claude (Anthropic)	Prompt: <i>no quiero que me haga esa autentificacion cuando apreto iniciar sesion con google, quiero que se haga el inicio normal de cualquier página, quiero que aparezca como la segunda foto y siga de forma normal como uno inicia sesion con google</i> Objetivo: Reemplazar la autenticación falsa por el popup real de Google Resultado: Se desistió de la idea porque no convencía la forma.

7. Fase 4: Prompts en Gemini

1. El Origen: Pasar de Privado a Público en GitHub

- **Objetivo:** Permitir que tus profesores (Boris Bugueño y Alejandro Paolini) y la plataforma Render puedan acceder a leer tu código.
- **Solución:** Ir a la pestaña **Settings**  del repositorio, bajar a la **"Danger Zone"**, seleccionar **"Change visibility"** y confirmar escribiendo el nombre de tu proyecto para transformarlo en **Público**.

2. El Arranque en Render (Configuración Inicial)

- **Objetivo:** Indicarle a Render qué comandos usar para construir y encender la app web creada originalmente en Google IDX.
- **Los comandos clave:**
 - **Build Command:** yarn install; yarn build (Para compilar los archivos de la app).
 - **Start Command:** Cambiar la casilla vacía que arrojaba el error **"Required"** por un comando de encendido válido.

3. La Batalla contra el Bloqueo de Vite (El error del "Host")

- **El Problema:** Al hacer clic en el link de Render, la página se quedaba en blanco mostrando el texto **"Solicitud bloqueada. Este host... no está permitido"** debido a las estrictas medidas de seguridad nativas de Vite.
- **El error intermedio (Command "start" not found):** Intentamos usar yarn start, pero el servidor arrojó que ese comando no existía para tu tipo de proyecto.
- **La Solución Definitiva en Render:** Utilice otra IA (Claude, para solucionar el problema porque lo intente con Gemini y no pude)

EN OTRO CHAT DE GEMINI

1. El Objetivo Inicial (La Bodega y la Tienda)

- **Tu consulta:** Querías conectar tu proyecto de GitHub llamado FreelanceFlow con la plataforma Render para subirlo a internet, advirtiéndote que necesitabas una explicación directa por tu perfil de ingeniera comercial.
- **Solución:** Identificamos la estructura de tus archivos y establecimos que GitHub actúa como tu bodega de código y Render como la vitrina pública.

2. Definición del Modelo de Negocio (¿Static o Web?)

- **Tu consulta:** Preguntaste qué significaba que un proyecto tuviera "un servidor detrás" y cómo saber cuál elegir.
- **Solución:** Usamos la analogía de un restaurante. Un **Static Site** es solo el comedor (diseño/folleto fijo), mientras que un **Web Service** incluye la cocina (servidor que procesa datos de clientes, inicios de sesión y bases de datos). Por archivos como .env.ejemplo, determinamos que tu proyecto requería obligatoriamente un **Web Service**.

3. Configuración de los Motores (Build vs Start)

- **Tu consulta:** Te generó confusión si debías usar npm install o npm start en la configuración de Render.
- **Solución:** Aclaramos que **Build Command** (npm install) es la fase de inversión/instalación de herramientas por una única vez, mientras que **Start Command** (npm start) es la operación diaria para encender el motor del sitio.

4. Seguridad y Credenciales (Variables de Entorno)

- **Tu consulta:** Revisamos el contenido de tu archivo .env.ejemplo (imagen image_8273c9.png) para saber qué poner en la sección *Environment* de Render.
- **Solución:** Descubrimos que el proyecto usa Inteligencia Artificial y definimos las dos llaves maestras que debías rellenar:
 - GEMINI_API_KEY: Tu acceso privado gestionado a través de Google AI Studio.
 - APP_URL: La dirección final de tu web. Diseñamos la estrategia de poner un enlace temporal (https://temporal.com) para que Render te dejara avanzar y luego reemplazarlo por el definitivo.

5. Resolución de Errores Operativos (El cambio a Vite)

- **Tu consulta:** El sistema arrojó un error rojo en pantalla: npm error Missing script: "start" (imagen image_8261c4.png).
- **Solución:** Detectamos que el proyecto fue programado con una herramienta moderna llamada **Vite** (por tu archivo vite.config.ts). Te di la solución inmediata de cambiar el comando de encendido en Render por **npm run dev** (o en su defecto npm run preview) para corregir las instrucciones de arranque de la aplicación. (NO PUDE HACERLO TAMPOCO CON ESTE CHAT)

8. Reflexión sobre el proceso

El uso de herramientas de IA generativa permitió acelerar significativamente el desarrollo del proyecto FreelanceFlow. La iteración constante de prompts fue clave para obtener resultados precisos, especialmente en la resolución de errores técnicos de despliegue. La combinación de Google AI Studio para generación de código, Stitch para prototipado y Claude para depuración resultó en un flujo de trabajo eficiente que permitió a una estudiante sin experiencia técnica previa desarrollar y desplegar una aplicación web funcional.